

Skill 04:-

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

OUTPUT 1 :-

```
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ nano Skill_04_1.c
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ gcc Skill_04_1.c -o Skill_04_1
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ ./Skill_04_1
Enter a command: ls -l | grep txt

--- Tokenizing Input ---

Tokens:
Token 1: ls           Type: WORD
Token 2: -l           Type: WORD
Token 3: |             Type: PIPE
Token 4: grep         Type: WORD
Token 5: txt          Type: WORD

Validation: Token stream is valid.

--- Debug Parsing Output ---
Total tokens = 5
```

OUTPUT 2 :-

Outputs of expected output, test syntax error, test empty command, test pipe error.

```
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ nano Skill_04_2.c
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ gcc Skill_04_2.c -o Skill_04_2
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ ./Skill_04_2
Enter a command: ls -l

Total tokens: 2

Syntax: VALID

--- Parse Tree ---
COMMAND
  COMMAND_NAME: ls
  ARGUMENT: -l

--- Execution Structure ---
Program: ls
Arguments: -l
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ ./Skill_04_2
Enter a command: | ls

Total tokens: 2

Syntax Error: Command cannot start with '|'.

--- Parse Tree ---
COMMAND
  PIPE
  ARGUMENT: ls
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ ./Skill_04_2
Enter a command:

Total tokens: 0

Syntax Error: Empty command.

--- Parse Tree ---
ROOT
  EMPTY_COMMAND
v_santosh@DESKTOP-E34M7RI:~/Skill_04.c$ ./Skill_04_2
Enter a command: ls | | grep txt

Total tokens: 5

Syntax Error: Two pipes cannot appear together.

--- Parse Tree ---
COMMAND
  COMMAND_NAME: ls
  PIPE
  PIPE
  ARGUMENT: grep
  ARGUMENT: txt
```

Code for Skill_04_1.c :-

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

typedef struct {
    char text[50];
    char type[20];
} Token;

int main() {
    char input[MAX];
    Token tokens[50];
    int count = 0;

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    printf("\n--- Tokenizing Input ---\n");

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && count < 50) {
        strcpy(tokens[count].text, token);

        if (strcmp(token, "|") == 0)
            strcpy(tokens[count].type, "PIPE");
        else if (strcmp(token, ">") == 0)
            strcpy(tokens[count].type, "REDIRECT");
        else if (strcmp(token, "<") == 0)
            strcpy(tokens[count].type, "REDIRECT");
        else if (strcmp(token, "&&") == 0)
            strcpy(tokens[count].type, "OPERATOR");
        else
            strcpy(tokens[count].type, "WORD");

        count++;
        token = strtok(NULL, " \\t\\n");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %-10s Type: %s\n",
            i + 1, tokens[i].text, tokens[i].type);
    }

    if (count == 0) {
        printf("\nValidation: Empty input!\n");
    } else {
        printf("\nValidation: Token stream is valid.\n");
    }

    printf("\n--- Debug Parsing Output ---\n");
    printf("Total tokens = %d\n", count);

    return 0;
}
```

Code for Skill_04_2.c :-

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

typedef struct {
    char text[50];
} Token;

int tokenize(char input[], Token tokens[]) {
    int count = 0;

    char *p = strtok(input, " \\t\\n");

    while (p != NULL && count < MAX) {
        strcpy(tokens[count].text, p);
        count++;
        p = strtok(NULL, " \\t\\n");
    }

    return count;
}

void printTree(Token tokens[], int count) {
    printf("\\n--- Parse Tree ---\\n");

    if (count == 0) {
        printf("ROOT\\n");
        printf("  EMPTY_COMMAND\\n");
        return;
    }

    printf("COMMAND\\n");

    for (int i = 0; i < count; i++) {
        if (strcmp(tokens[i].text, "|") == 0) {
            printf("  PIPE\\n");
        } else if (strcmp(tokens[i].text, ">") == 0) {
            printf("  REDIRECT_OUT\\n");
        } else if (strcmp(tokens[i].text, "<") == 0) {
            printf("  REDIRECT_IN\\n");
        } else if (i == 0) {
            printf("  COMMAND_NAME: %s\\n", tokens[i].text);
        } else {
            printf("  ARGUMENT: %s\\n", tokens[i].text);
        }
    }
}

int validate(Token tokens[], int count) {
    if (count == 0) {
        printf("\\nSyntax Error: Empty command.\\n");
        return 0;
    }

    if (strcmp(tokens[0].text, "|") == 0) {
        printf("\\nSyntax Error: Command cannot start with '|'.\\n");
        return 0;
    }

    if (strcmp(tokens[count - 1].text, "|") == 0) {
        printf("\\nSyntax Error: Command cannot end with '|'.\\n");
    }
}
```

```

    if (strcmp(tokens[count - 1].text, "|") == 0) {
        printf("\nSyntax Error: Command cannot end with '|'.\n");
        return 0;
    }

    for (int i = 0; i < count - 1; i++) {
        if (strcmp(tokens[i].text, "|") == 0 &&
            strcmp(tokens[i + 1].text, "|") == 0) {
            printf("\nSyntax Error: Two pipes cannot appear together.\n");
            return 0;
        }
    }

    printf("\nSyntax: VALID\n");
    return 1;
}

void executionStructure(Token tokens[], int count) {
    printf("\n--- Execution Structure ---\n");

    if (count == 0)
        return;

    printf("Program: %s\n", tokens[0].text);

    printf("Arguments: ");

    for (int i = 1; i < count; i++) {
        if (strcmp(tokens[i].text, "|") == 0) {
            printf("\nPipeline detected\n");
            printf("Next command: ");
        } else {
            printf("%s ", tokens[i].text);
        }
    }

    printf("\n");
}

int main() {
    char input[MAX];
    Token tokens[MAX];

    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    int count = tokenize(input, tokens);

    printf("\nTotal tokens: %d\n", count);

    if (validate(tokens, count)) {
        printTree(tokens, count);
        executionStructure(tokens, count);
    } else {
        printTree(tokens, count);
    }

    return 0;
}

```